Circuit Diagram Images to Netlists: State of the Art (2024–2026)

Nov 5, 2026 · @Nikita

Summary

Hybrid pipelines are the state of the art; pure vision-language models (VLMs) still can't read circuit topology reliably. The best 2024–2026 systems combine four parts:

- a trained detector for components;
- classical or learned image processing for wires;
- OCR for text;
- an LLM or VLM only for final assembly and checking.

The hardest sub-problem is connectivity. Above all, that means telling a connected junction from two wires that merely cross.

1. How well do end-to-end LLMs/VLMs do?

Not well enough. Use VLMs as a reader, verifier and assembler, not as the primary topology extractor.

- **Masala-CHAI (NYU, 2024–25)** analysed GPT-4o's failure modes in detail:
  - it assumed intersecting wires were connected when they weren't;
  - it confused MOSFET drain, gate and source;
  - it mishandled differential input and output pairs;
  - it confused NMOS with PMOS and missed components.
- **AMSbench (2025)** is more favourable on simple analog circuits. Connection-identification F1 was 0.88 for Gemini-2.5-pro, 0.65 for GPT-4o and 0.71 for Claude-3.7-sonnet.
- **OmniSch (2026)** is the most sobering result. It tested 1,854 real-world PCB schematics:
  - The best LMM reached only 27.44% net-detection F1, and results degraded as net count grew.
  - Models could name the symbols present but not place them. IoU-based symbol detection stayed below 0.3% for every LMM.
  - A classical pipeline (YOLO11 + PaddleOCR + rule-based connectivity) was the stronger localization baseline.

    - Agentic crop-and-zoom search improved results once a target region was supplied. In practice: a detector proposes regions, and the VLM reads them.

2. Parsing components

A fine-tuned YOLO detector plus a separate orientation model is the standard. OCR and a VLM split the text work.

Detection. Almost every recent system fine-tunes a YOLO-family detector on schematic symbols.

- SINA chose YOLOv11m over YOLOv11n for its higher mAP, despite the slower inference.
- Image2Net reports mAP@50 of 98.38% and mAP@50-95 of 74.31% across 22 device classes.
- Hu et al. use a Swin-Transformer-based detector instead.

Orientation as a separate model. Orientation decides which pin is which, so it matters a great deal.

- Netlistify uses YOLOv8 for detection and ResNet for orientation. Splitting them improved detection accuracy.
- Image2Net uses MobileNetV2 to classify orientation and mirroring. Once one port is fixed, it orders the remaining ports clockwise. Its classifiers score about 95–99.9% accuracy.
- AMSNet instead folds orientation into the detector's class label.

Pins on rectangular ICs (PCB-level schematics). Box-shaped parts with arbitrary pin layouts need more than bounding boxes.

- Pose models win. In SINA, YOLOv8m-pose reached mAP50–95 of 94%, with 94% precision and 98% recall.
- SINA finds pins by skeletonizing wires after masking text. It then clusters skeleton endpoints that touch dilated component boxes.
- That method scored 95.3% pin F1, against 43.6% for a heuristic edge scan.

Text: OCR and VLM, kept separate.

- VLMs struggle to locate text annotations even in simple circuits. SINA therefore lets OCR (EasyOCR) read the text and the VLM interpret it.
- OCR input is preprocessed with contrast enhancement, denoising and adaptive thresholding.
- Each text item is matched to the nearest component.

- For PCB schematics, text is OCR'd and then inpainted out before component detection.

VLM as a cross-check. SINA compares the VLM's component counts and types against YOLO's detections. The result is a confidence score, and mismatches are flagged for review.

3. Parsing edges and connections

Connectivity is where pipelines fail: accurate detection does not guarantee a correct netlist. The work splits into five sub-problems.

3a. Isolating the wires

The standard trick is to mask out detected components and treat what remains as wiring. Approaches range from simple pixel rules to learned segmentation.

- **Pixel connectivity (CCL / BFS).** This is the simplest robust option.
  - SINA runs connected-component labeling on the masked image; OpenCV's `connectedComponentsWithStats` does this.
  - It drops stubs and gaps and keeps regions that touch two or more components.
  - It merges equivalent nodes, such as multiple ground symbols.
  - AMSNet used a breadth-first search over black pixels instead.
- **Binarize + skeletonize.** Image2Net is a good template:
  - Otsu thresholding;
  - Zhang–Suen skeletonization to 1-pixel wires;
  - removing regions smaller than 1/10 of the largest one, which strips text;
  - removing devices; each remaining connected region is then a net.
- **Line detection.** Masala-CHAI used deep Hough-transform line priors. It grouped segments whose endpoints fell within 40 pixels into one net. That fixed radius is a known weakness.
- **Learned wire segmentation.** This is the better option for scans and photos.
  - AMSnet 2.0 uses a U-Net for wire masks, then splits and merges at intersections to form nets.
  - Its training trick: draw random rectangles and text over clean schematics while keeping the wire labels unchanged.

3b. Junctions vs. crossings

This is the single most important topology decision, and drawing conventions differ between sources.

- **Image2Net treats it as detection.** It annotated 28,195 crossings of three types: bridge, flat and dot.
- **It infers the convention per diagram:**
  - If any bridge (hop) is present, bridges are jumpers and the other crossing types are connections.
  - If there are no bridges but both dots and flat crossings appear, the flat crossings are jumpers.
- **Jumpers are resolved geometrically.** Ports are numbered clockwise, and each port connects only to the opposite one.
- **The payoff is large.** The success rate was 80.77%, versus 35.58% with AMSNet's crossing logic and 46.15% with Masala-CHAI's. AMSNet assumes crossings never connect; Masala-CHAI assumes they always do.
- **SINA v2 does it geometrically.**
  - It extracts line segments with morphology and contours, and checks only perpendicular pairs for intersections.
  - For each crossing it builds two images, each keeping one wire continuous and marking the other's endpoints with virtual bridges.
  - It runs connectivity on both and merges the two netlists.
  - Result: 87% crossing accuracy and 100% precision.

3c. Learned connectivity

Two systems replace tracing rules with a trained model.

- **Netlistify** uses a windowed transformer for connectivity. Fixed-size convolutions miss thin, sparse wires in low-resolution images. It trains on synthetic schematics exported from a commercial EDA tool, then fine-tunes on textbook images.
- **Hu et al.** treat ports as graph nodes and predict links with a graph attention variational autoencoder (SGL-VGAE). SINA's authors note it needs precise terminal detection and struggles on cluttered schematics.

3d. Port-to-net assignment

Ports are where a device's bounding box meets a wire; the orientation classifier then sets their order. All ports touching the same wire region share a net.

Post-processing handles special cases:

- merge all nets touching ground (GND) symbols into one;
- merge the nets on each jumper's paired ports;
- resolve temporary bulk-terminal ports.

3e. Keeping geometry

An editable schematic needs more than a netlist; it needs positions. AMSnet 2.0's segmentation recovers them, so it can rebuild schematics in OpenAccess format.

4. Assembly, output and verification

The LLM assembles the final netlist from structured inputs; correctness is then checked with graph metrics and simulation.

LLM as assembler. SINA gives GPT-4o three inputs: the OCR'd designators, the component-to-node map and the original image. GPT-4o returns the SPICE netlist.

Masala-CHAI improves the LLM's read with two tricks:

- It overlays net numbers on the image in red. The prompt says to pick a MOSFET's source as the net nearest its arrow.
- A final LLM pass corrects common errors such as floating nets.

Metrics. Plain accuracy hides topology errors, so recent work compares netlists as graphs.

- Image2Net's **Netlist Edit Distance (NED)** is the most rigorous option.
  - The netlist becomes a heterogeneous graph whose nodes are devices and nets.
  - Edges are typed by terminal, for example PMOS_Gate.
  - NED is the graph edit distance divided by the total count of devices, nets and ports.
  - It was designed because Masala-CHAI's metric ignores port identity, so some wrong netlists scored as correct.
- SINA uses graph isomorphism on a bipartite component–node graph, plus Ngspice simulation. On 40 circuits, 92.5% matched topologically and 90.0% matched the ground-truth simulation within 5%.

Headline results

System	Result	Compared against
SINA v2 (TCAD, Jul 2026)	96.67% overall accuracy, 2.72x Masala-CHAI	40 IC schematics, Masala-CHAI at 35.5%

System	Result	Compared against
Image2Net (ISEDA 2025)	80.77% success rate, 0.116 average NED	Own 104-pair benchmark

The two results are not directly comparable. SINA's 40-circuit set left out schematics Masala-CHAI can't handle, such as those with wire crossings.

5. Datasets and synthetic data

Public datasets now exist for analog IC and PCB schematics. The most reusable idea is rendering labelled images straight from EDA source files.

Dataset	Year	Size	Notable labels
OmniSch	2026	1,854 real PCB schematics; 109.9K symbols (3.4K types), 245.4K pins	Symbols, pins, text, spatial net graphs
Image2Net	2025	2,914 diagrams, 84,195 annotations	Devices, crossings (bridge/flat/dot), orientation
AMSnet 2.0	2025	2,686 circuits	Spectre netlists, OpenAccess schematics, positions
Netlistify	2025	~100,000 images	YOLO boxes + orientation, net labels, HSPICE netlists
Masala-CHAI	2024	~7,500 schematics from 10 textbooks	SPICE netlists + captions
CGHD (hand-drawn)	2021+	2,424 images, 59 classes	Boxes, text, segmentation maps

Copy OmniSch's data strategy. It gathered EAGLE schematics from SparkFun, Arduino, Adafruit and GitHub. A custom engine then rendered each XML file to an image with pixel-aligned labels, including net connectivity. Netlistify does something similar with synthetic exports from a commercial tool.

The same approach should work with KiCad files. That is my suggestion, not something a paper tested.

6. Hand-drawn circuits (optional)

Hand-drawn input works with the same pipeline shape, at lower accuracy.

- **Baseline.** Bayer et al. (RPTU/DFKI, ICDAR 2024) built an end-to-end pipeline on the public CGHD dataset.
- **Edges.** Blob detection on the binary stroke map finds wires. Each blob touching two objects becomes an edge, and post-processing resolves wire hops.
- **Why it works.** Stroke segmentation reaches 98.16% pixel accuracy.
- **Accuracy gap.** A 2026 OST student thesis reports detection mAP@50–95 above 95% on digital schematics, but only above 80% on hand-drawn ones.

7. Practical takeaway: a recommended stack

This is a defensible pipeline for digital schematic images today, in order:

1. **Components:** a YOLO11 detector for symbols, junction dots and crossing types. Add a pose model for IC pins and a separate orientation classifier.
2. **Text:** PaddleOCR or EasyOCR. Inpaint the text out of the image before extracting wires.
3. **Wires:** for clean digital images, Otsu + skeleton + connected-component labeling. For scans and photos, use a U-Net wire mask.
4. **Crossings:** classify junctions and crossings explicitly. Infer each diagram's convention the way Image2Net does.
5. **Assembly:** order ports from orientation with rules. Use a VLM only to assign designators and values and to flag inconsistencies.
6. **Validation:** check structure with NED or graph isomorphism, then run a SPICE simulation as a smoke test.

Train on synthetic renders from EDA source files, then fine-tune on real images. Both Netlistify and OmniSch found this necessary.

Sources

- SINA: A Fully Automated Circuit Schematic Image to Netlist Generator (IEEE TCAD, arXiv 2607.01609, Jul 2026)
- SINA: A Circuit Schematic Image-to-Netlist Generator (DATE 2026, arXiv 2601.22114)
- Image2Net: Datasets, Benchmark and Hybrid Framework (ISEDA 2025, arXiv 2508.13157)
- OmniSch: A Multimodal PCB Schematic Benchmark (arXiv 2604.00270, 2026)
- Masala-CHAI: A Large-Scale SPICE Netlist Dataset (arXiv 2411.14299)
- Netlistify: Transforming Circuit Schematics into Netlists with Deep Learning (MLCAD 2025) · dataset

- AMSNet: Netlist Dataset for AMS Circuits (arXiv 2405.09045)
- AMSnet 2.0: A Large AMS Database with AI Segmentation for Net Detection (ICLAD 2025)
- AMSbench: Evaluating MLLM Capabilities in AMS Circuits (arXiv 2505.24138)
- Hu et al., Parsing Netlists of Integrated Circuits from Images via Graph Attention Network (Sensors 2024)
- Bayer et al., Modular Graph Extraction for Handwritten Circuit Diagram Images (ICDAR 2024)
- OST thesis: Automated Component Detection and Connectivity Analysis in Circuit Schematics (2026)